ANALYSIS REPORT

Training Configuration Analysis Report

Feedforward Transmission Error Baseline

Styled PDF edition generated from the canonical Markdown analysis report for improved readability, section hierarchy, and table presentation.

Overview

This document explains the main training and dataset configuration choices used in the current feedforward Transmission Error baseline.

The report has four goals

1. explain what each relevant configuration entry means;
2. explain what each entry actually does in the current implementation;
3. explain the practical effect of each entry on runtime, convergence, and final metrics;
4. compare the executed proof configuration against the repository baseline and against more aggressive workstation-oriented variants.

This document should be read together with:

- `config/feedforward_network_training_trial.yaml`
- `config/feedforward_network_training.yaml`
- `config/feedforward_network_training_high_density.yaml`
- `config/feedforward_network_training_high_epoch.yaml`
- `config/feedforward_network_training_high_compute.yaml`
- `config/dataset_processing.yaml`
- `training/train_feedforward_network.py`
- `training/transmission_error_datamodule.py`
- `training/transmission_error_regression_module.py`
- `models/feedforward_network.py`
- `output/feedforward_network/te_feedforward_trial/training_test_metrics.yaml`

What The Current Training Pipeline Actually Does

The current baseline is a point-wise feedforward regression model.

The dataset loader first builds directional TE curves from validated CSV files. Each point of each curve is represented by five physical input features:

1. output angular position in degrees
2. input speed in rpm
3. input torque in Nm
4. oil temperature in degrees
5. motion direction flag

The model does **not** currently ingest raw encoder streams, harmonic coefficients, or temporal memory states. It learns a direct mapping: `[angular_position, speed, torque, temperature, direction] -> transmission_error` This is why `model.input_size` is `5`. It is not an arbitrary small network input. It is the exact number of features the current dataset pipeline exposes at each sampled point.

Dataset Processing Config

`paths.dataset_root`

Meaning

- root folder of the validated TE CSV dataset.

  What it does:

- tells the dataset utilities where to collect the CSV files.

  Effect on training:

- no direct effect on optimization quality by itself;
- wrong path means the workflow cannot start;
- moving to a different dataset root can change everything if the data distribution changes.

`directions.use_forward_direction`

Meaning

- include forward curves.

  What it does:

- enables directional samples built from the forward TE columns.

  Effect on training:

- more data;
- better coverage of motion asymmetry;
- removing it would reduce data diversity and could bias the model.

`directions.use_backward_direction`

Meaning

- include backward curves.

  What it does:

- enables directional samples built from the backward TE columns.

  Effect on training:

- same reasoning as forward direction;
- important because backlash and hysteresis-related behavior can differ by direction.

`split.validation_split`

Meaning

- percentage of source files reserved for validation.

  What it does:

- the datamodule splits files, not individual points, so full directional curves from a file stay together.

  Effect on training:

- too small: unstable validation signal;
- too large: less training data;
- current value `0.2` is reasonable for a first baseline.

`split.test_split`

Meaning

- percentage of source files reserved for held-out test.

  What it does:

- keeps a final evaluation set fully separate from training and validation.

  Effect on training:

- does not affect gradient updates directly;
- improves credibility of the final result;
- reduces train-set size slightly but gives a more honest generalization estimate.

`split.random_seed`

Meaning

- seed for reproducible file-level split generation.

  What it does:

- makes train/validation/test partitioning repeatable.

  Effect on training:

- does not improve the model by itself;
- makes comparisons across runs fairer;
- one seed is not enough for a complete conclusion, but it is necessary for reproducibility.

Training Config - Paths And Experiment

`paths.dataset_config_path`

Meaning

- which dataset-processing YAML the training script should use.

  What it does:

- connects the training workflow to the selected dataset split and data-root definition.

  Effect on training:

- indirect but critical;
- if this path changes, the exact train/validation/test split and dataset source can also change.

`paths.output_root`

Meaning

- root folder where checkpoints, logs, and reports are written.

  What it does:

- defines the artifact storage location.

  Effect on training:

- no direct numerical effect;
- important for experiment traceability.

`experiment.run_name`

Meaning

- name of the current experiment folder.

  What it does:

- isolates artifacts for one run from the others.

  Effect on training:

- no direct effect on convergence;
- essential for comparing runs cleanly.

`experiment.model_type`

Meaning

- architecture selection key.

  What it does:

- is resolved by `models/model_factory.py`.

  Effect on training:

- currently only `feedforward` is implemented;
- in the future this field will determine whether the model is point-wise, recurrent, or physics-informed.

Training Config - Dataset Section

`dataset.curve_batch_size`

Meaning

- number of directional curves loaded together before they are flattened into a point batch.

  What it does:

- each selected curve is sub-sampled according to `point_stride` and `maximum_points_per_curve`;
- then all selected points are concatenated into one batch.

  Effect on training:

- larger value:
  - more points per optimizer step;
  - smoother gradients;
  - higher memory cost;
  - potentially better GPU utilization.
- smaller value:
  - less memory pressure;
  - noisier gradients;
  - slower effective throughput.

  Important nuance:

- here "batch size" is not plain point count;
- its real cost depends strongly on how many points survive the sampling stage.

`dataset.point_stride`

Meaning

- sub-sampling interval applied along each curve.

  What it does:

- only every `N`-th point is kept.

  Effect on training:

- higher stride:
  - fewer points;
  - faster training;
  - lower memory cost;
  - more information loss;
  - weaker harmonic/detail fidelity.
- lower stride:
  - denser data;
  - better access to local TE structure;
  - heavier runtime and memory cost.

    This is one of the most important levers in the whole config.

`dataset.maximum_points_per_curve`

Meaning

- optional hard cap on sampled points retained from each curve after stride selection.

  What it does:

- if too many points remain, they are reduced to a uniformly spread subset.

  Effect on training:

- useful to control runtime;
- risky if set too low because it can remove important local structure;
- `null` is better when the goal is best-quality offline learning and the hardware can sustain it.

`dataset.num_workers`

Meaning

- number of PyTorch DataLoader worker processes.

  What it does:

- parallelizes data loading on the CPU.

  Effect on training:

- more workers can improve throughput;
- too many workers on Windows can reduce stability or even make training slower;
- best value is hardware-dependent and should be measured.

`dataset.pin_memory`

Meaning

- whether DataLoader output should be pinned in host memory.

  What it does:

- speeds host-to-device transfer when using GPU.

  Effect on training:

- useful on GPU runs;
- typically not useful on CPU-only runs;
- may slightly increase host memory usage.

Training Config - Model Section

`model.input_size`

Meaning

- number of input features per sampled point.

  What it does:

- defines the first linear layer input dimension.

  Effect on training:

- must match the dataset feature count exactly;
- a larger value is only meaningful if you actually add real new features.

  Current judgment:

- `5` is correct for the present dataset pipeline;
- it is not "too small" in itself;
- the real question is whether the **feature set** is sufficient, not whether the integer looks small.

`model.output_size`

Meaning

- number of output regression values per point.

  What it does:

- defines the final layer output width.

  Effect on training:

- current value `1` is correct because the model predicts one scalar TE value.

`model.hidden_size`

Meaning

- hidden-layer widths of the MLP.

  What it does:

- controls model capacity.

  Effect on training:

- larger hidden layers:
  - more expressive model;
  - more memory and compute;
  - higher overfitting risk;
  - more potential to fit subtle nonlinearities.
- smaller hidden layers:
  - cheaper and simpler;
  - may underfit.

    Current baseline `128-128-64` is a sensible middle ground for 5 inputs and one scalar output.

`model.activation_name`

Meaning

- nonlinearity between linear layers.

  What it does:

- introduces nonlinear modeling capacity.

  Effect on training:

- `GELU` is a good modern default for smooth regression;
- `ReLU` would be cheaper but slightly harsher;
- this choice matters, but less than data coverage and sampling density.

`model.dropout_probability`

Meaning

- dropout rate after hidden activations.

  What it does:

- randomly drops hidden activations during training.

  Effect on training:

- helps reduce overfitting;
- too much dropout can slow or weaken fitting;
- `0.10` is mild and reasonable.

`model.use_layer_norm`

Meaning

- enable `LayerNorm` after hidden linear layers.

  What it does:

- stabilizes hidden activation statistics.

  Effect on training:

- usually helps optimization stability;
- modest runtime overhead;
- reasonable choice for a tabular-style MLP baseline.

Training Config - Optimization Section

`training.learning_rate`

Meaning

- optimizer step size.

  What it does:

- controls how aggressively weights are updated.

  Effect on training:

- too high: unstable training, oscillation, divergence;
- too low: very slow progress;
- `0.001` is a strong baseline value for AdamW on this kind of MLP.

`training.weight_decay`

Meaning

- `AdamW` weight regularization.

  What it does:

- discourages unnecessarily large weights.

  Effect on training:

- helps generalization a bit;
- too much can underfit;
- `1e-4` is conservative and reasonable.

`training.min_epochs`

Meaning

- minimum epoch count before early stopping can terminate training.

  What it does:

- guarantees the model sees at least some minimum amount of optimization.

  Effect on training:

- too low: a short lucky fluctuation can stop training too early;
- too high: wasted runtime if the model is already saturated.

`training.max_epochs`

Meaning

- hard upper limit on epochs.

  What it does:

- puts a ceiling on total runtime.

  Effect on training:

- too low: undertraining;
- too high: safe if early stopping works, but expensive.

`training.patience`

Meaning

- number of epochs without meaningful validation improvement allowed before early stopping.

  What it does:

- determines how long the run can continue after the validation metric plateaus.

  Effect on training:

- low patience:
  - faster experiments;
  - higher risk of stopping before the model fully settles.
- high patience:
  - safer for deeper convergence;
  - more compute cost.

`training.min_delta`

Meaning

- minimum improvement required for early stopping to count as real progress.

  What it does:

- filters out tiny metric fluctuations.

  Effect on training:

- too high: early stopping becomes too aggressive;
- too low: training can continue on noise.

`training.log_every_n_steps`

Meaning

- how often Lightning logs step-level information.

  What it does:

- controls metric/logging frequency.

  Effect on training:

- no direct effect on model quality;
- too frequent logging can add overhead.

`training.fast_dev_run`

Meaning

- Lightning debug shortcut.

  What it does:

- runs a tiny smoke test rather than a real training campaign.

  Effect on training:

- should remain `false` for real experiments.

`training.deterministic`

Meaning

- ask Lightning/PyTorch to prefer deterministic execution.

  What it does:

- improves reproducibility at the cost of some performance.

  Effect on training:

- useful when strict repeatability matters;
- may reduce throughput.

Interpreting The Executed Trial Choices

The proof run used

- `curve_batch_size: 8`
- `point_stride: 200`
- `maximum_points_per_curve: 200`
- `num_workers: 0`
- `pin_memory: false`
- `min_epochs: 3`
- `max_epochs: 12`
- `patience: 3`

  Why these choices were made:

- reduce runtime on a first end-to-end verification;
- keep memory pressure low;
- confirm that the new train/validation/test workflow works;
- generate a checkpoint and report quickly.

  Why they are **not** ideal for a final benchmark:

- `point_stride: 200` is very sparse;
- `maximum_points_per_curve: 200` removes a lot of curve detail;
- `3-12` epochs are enough for a trial, but not for a serious study;
- `num_workers: 0` and `pin_memory: false` do not exploit a strong workstation.

  Conclusion:

- good trial settings;
- not the right target configuration for the strongest offline benchmark.

Comparing Trial, Baseline, And Workstation Variants

All configurations below use the same dataset root and the same `70/20/10` train/validation/test split.

Configurations

Campaign Summary

Config	Status	Main Intent
`trial`	executed	quick pipeline verification
`baseline`	current repo default	first serious baseline
`high_density`	proposed	denser curve sampling
`high_epoch`	proposed	longer convergence window
`high_compute`	proposed	workstation-scale offline benchmark

Data Pipeline Settings

Config	Curve Batch	Point Stride	Max Points/Curve	Workers	Pin Memory
`trial`	8	200	200	0	false
`baseline`	2	20	null	4	true
`high_density`	4	10	null	8	true
`high_epoch`	2	20	null	8	true
`high_compute`	6	5	null	8	true

Model And Schedule Settings

Config	Hidden Layers	Epoch Budget	Patience
`trial`	128-128-64	3-12	3
`baseline`	128-128-64	10-150	20
`high_density`	128-128-64	15-180	25
`high_epoch`	128-128-64	20-250	35
`high_compute`	256-256-128-64	20-250	35

Trial

Strengths

- very fast;
- enough to validate the workflow;
- already produced credible held-out metrics.

Weaknesses:

- not dense enough to represent the full detail of each TE curve;
- too short to conclude that the optimization schedule is fully adequate.

Use it for:

- code verification;
- smoke tests;
- quick environment checks.

Baseline

Strengths

- much better data density than the trial;
- still conservative enough to remain robust on Windows;
- good default starting point for a real benchmark.

Weaknesses:

- still leaves performance on the table if the workstation is strong;
- may converge later than `150` epochs in some cases;
- worker count is still conservative.

Use it for:

- the first real benchmark you would trust more than the trial.

High Density

What changes

- lower `point_stride` from `20` to `10`;
- increase `curve_batch_size` from `2` to `4`;
- raise `num_workers` to `8`.

Expected effect:

- much denser exposure to the TE curve shape;
- better chance to capture local structure and harmonic detail;
- heavier GPU memory and longer epoch time;
- likely better offline accuracy if the model can use the extra information.

Main risk:

- higher compute cost without guaranteed payoff if the current feature set is the main bottleneck rather than sampling density.

High Epoch

What changes

- keeps baseline data density;
- extends training window to `20-250` epochs;
- increases patience to `35`.

Expected effect:

- better chance to fully exploit the current model and current data representation;
- cheaper than increasing both model and data density at once;
- good way to test whether the baseline is currently undertrained.

Main risk:

- if the model is already capacity-limited, more epochs alone may give only marginal gains.

High Compute

What changes

- much denser data with `point_stride: 5`;
- larger `curve_batch_size: 6`;
- larger hidden layers `256-256-128-64`;
- longer epoch window;
- slightly smaller learning rate for stability.

Expected effect:

- strongest offline benchmark among the listed options;
- better use of a powerful GPU and CPU loader pipeline;
- more capacity to learn fine nonlinear structure.

Main risk:

- more overfitting potential;
- less aligned with strict PLC-friendly simplicity;
- more sensitive to runtime stability and DataLoader throughput on Windows.

Which Choices Are Good For An Effective Training Campaign

Clearly Good Choices

- `input_size: 5`

  Because it correctly matches the current feature set.
- `output_size: 1`

  Because the target is scalar TE.
- `hidden_size: 128-128-64`

  Good first baseline size.
- `activation_name: GELU`

  Reasonable regression default.
- `dropout_probability: 0.10`

  Mild regularization.
- `use_layer_norm: true`

  Good stability choice.
- `learning_rate: 0.001`

  Reasonable AdamW starting point.
- `weight_decay: 1e-4`

  Reasonable regularization baseline.
- `validation_split: 0.2` and `test_split: 0.1`

  Good for honest evaluation.

Good For Trial, But Not Enough For Final Benchmark

- `point_stride: 200`
- `maximum_points_per_curve: 200`
- `min_epochs: 3`
- `max_epochs: 12`
- `patience: 3`
- `num_workers: 0`
- `pin_memory: false`

  These are verification-oriented settings, not "push the workstation" settings.

Most Defensible Next Step

If the goal is a serious next run, the most defensible progression is:

1. run the **baseline** first;
2. if runtime is acceptable, try **high_density**;
3. if convergence still looks incomplete, try **high_epoch**;
4. only then try **high_compute** as the strongest offline benchmark.

   That sequence isolates whether the main gains come from:

- more data density;
- more optimization time;
- or more model/data compute together.

Trial Metrics For Context

The executed trial run produced

- validation MAE: `0.003757 deg`
- validation RMSE: `0.004555 deg`
- test MAE: `0.003583 deg`
- test RMSE: `0.004295 deg`

This confirms that even the lightweight trial is stable. The question now is not "does the pipeline work?" because it does. The real question is "which heavier configuration gives the best accuracy-versus-cost tradeoff on the available workstation?"

Final Recommendation

My recommendation is

- do **not** treat the trial config as the final training setup;
- treat the current repository baseline as the first serious benchmark;
- use the proposed workstation configs to explore the tradeoff space in a structured way rather than just making everything bigger at once.

If you want only one immediate next configuration to run, choose:

- `config/feedforward_network_training_high_density.yaml`

Why:

- it increases information density strongly;
- it still keeps the model architecture fixed, so the comparison stays clean;
- it is the most informative next test if the workstation is genuinely powerful.

If that run is stable and materially better, the next most useful follow-up is:

- `config/feedforward_network_training_high_compute.yaml`

That will tell us whether the current bottleneck is mainly data density or whether extra model capacity is also worth paying for.